

OpenQARP (Quarp) 実践導入・活用ガイド

富士通量子アプリケーション研究開発基盤のアーキテクチャと
QAOA / XY-QAOA 実装プロトコル

富士通インターンシップ研究プロジェクト
量子コンピューティング・ブラックボックス最適化 (BBO) チーム

2026 年 9 月 (OpenQARP v0.1.0 公式公開版準拠)

概要

本技術ガイドは、富士通株式会社が 2026 年 9 月 15 日にオープンソースソフトウェア (OSS) として世界に一般公開した次世代量子アプリケーション開発基盤 **OpenQARP** (Open Quantum Application Research Package, モジュール名: **quarp**) のアーキテクチャ、モジュール構成、および量子近似最適化アルゴリズム (QAOA) の実践的プログラミング手法を包括的にまとめた技術解説書である。OpenQARP は、70 種以上の構成要素 (Composable Blocks) と 20 種以上の既成アルゴリズムを標準搭載し、C++ 超高速コアエンジン (**quarpx**) による卓越したシミュレーション性能を備える。本稿では、OpenQARP の基本思想から環境構築、演算子定義、回路合成、標準 QAOA の実行方法に加え、本研究の中核である 1-hot 制約保存型 **FM-XY-QAOA** への拡張実装手順までを完全なコード例とともに詳解する。

目次

1	はじめに：OpenQARP 公開の意義と本研究での位置づけ	1
1.1	富士通によるオープンソース公開の概要	1
1.2	本研究 (触媒材料 BBO・FM-XY-QAOA) における採用意義	2
2	システムアーキテクチャとディレクトリ構造	2
3	インストールと環境設定	3
3.1	動作要件	3
3.2	インストール手順	3
3.3	インポートと ABI 適合性検証	3
4	コア機能と API リファレンス	3
4.1	ハミルトニアン定義：QubitOperator	3
4.2	回路ビルディングブロック：Block と CompositeBlock	4
4.3	測定プリミティブと実行エンジン	4
5	OpenQARP による標準 QAOA の実装	5
5.1	QUBO 問題からの QAOA 実行	5
6	発展：制約付き最適化と「XY-QAOA」のカスタム構築	5

6.1	XY ミキサーブロックの設計数理	6
6.2	XY-QAOA クラスの完成	7
7	BBO（触媒・機能性材料探索）への統合と今後の検証計画	7
7.1	既存 Qiskit 実装からのリプレイス手順	7
7.2	性能比較・ベンチマークのロードマップ	8
8	まとめ	8

1 はじめに：OpenQARP 公開の意義と本研究での位置づけ

1.1 富士通によるオープンソース公開の概要

2026 年 9 月 15 日、富士通株式会社は産業界および学術界における量子アルゴリズムの実用化研究を劇的に加速させるため、量子アプリケーション開発フレームワーク **OpenQARP** (Open Quantum Application Research Package) を GitHub 上でオープンソース (Apache License 2.0) として公開した^{*1}。

従来の量子ソフトウェアフレームワーク (Qiskit, PennyLane, Cirq 等) は、低レイヤの量子回路記述や抽象的な量子化学計算には優れている一方、産業課題特有のハイブリッド量子・古典アルゴリズム (VQA/QAOA) の構築、回路ブロックのモジュール的再構成、および古典シミュレーションの大規模化において、コード記述の冗長化や実行速度のボトルネックが存在していた。OpenQARP はこれらの課題を打破すべく開発された。

OpenQARP のコアバリュー

- **モジュール型構成 (Composable Blocks)** : 70 種以上のビルディングブロックをレゴブロックのように組み合わせ、独自の変分回路やミキサー回路を数行で合成可能。
- **C++ バックエンドによる超高速実行 (qarpx)** : 内部の基底ベクトル演算やパウリ代数は、高精度・高並列化された C++ コアモジュール qarpx によって直接実行され、Python オーバーヘッドを完全排除。
- **ハイブリッド基盤親和性**: NVIDIA CUDA-Q プラットフォームや富士通の高性能量子シミュレータ、将来の超伝導・中性原子実機へのシームレスな移行をサポート。

1.2 本研究（触媒材料 BBO・FM-XY-QAOA）における採用意義

現在進めている「Factorization Machine と量子近似最適化を融合した材料ブラックボックス最適化 (FM-QAOA)」において、OpenQARP を採用することは以下の決定的なメリットを持つ：

1. **インターン成果としての完璧なシナジー**: 富士通の最新成果である OpenQARP を、材料設計という極めて実用性の高いドメインにいち早く適用した「世界初の先駆的実証事例」となる。
2. **XY ミキサーの実装容易性**: OpenQARP の CompositeBlock アーキテクチャにより、従来の Qiskit では複雑な回路手組みが必要だった「部分空間保存スワップ演算子 (XY ミキサー)」を、ネイティブなブロックとして極めてエレガントに定義できる。

^{*1} プレスリリース: <https://global.fujitsu/ja-JP/pr/news/2026/09/15-01>, GitHub: <https://github.com/OpenQARP/openqarp>

3. **BBO サイクルの劇的高速化:** 各サイクルで反復されるオンラインサンプリングにおいて, C++ コアによる高速処理がシミュレーション時間を大幅に短縮する.

2 システムアーキテクチャとディレクトリ構造

OpenQARP は, クリーンで階層的なモジュール設計を採用している. 主要なサブパッケージとその役割を表 1 に示す.

表 1: OpenQARP の主要パッケージ構成と役割

サブパッケージ名	インポートパス	主要な役割・機能
algorithms	qarp.algorithms	QAOA, VQE, VQD, QPE, Grover などの完成アルゴリズム
blocks	qarp.blocks	量子回路の最小構成単位 (単一ゲート, H 層, Cost 層, Mixer 層, Ansatz)
operators	qarp.operators	OpenFermion 互換・C++ 高速パウリ演算子 (QubitOperator)
engines	qarp.engines	実行エンジン (高速 C++ シミュレータ QarpEngine, CUDA-Q Engine)
optimizers	qarp.optimizers	パラメータ最適化器 (ScipyOptimizer, 解析的勾配対応)
graphs	qarp.graphs	NetworkX グラフとハミルトニアンとの相互変換・トポロジ解析
devices	qarp.devices	実機トポロジ, カップリングマップ, ノイズモデルの定義
plotting	qarp.plotting	エネルギー収束履歴, ブロッホ球, 状態分布の可視化

重要: パッケージ名とインポート名の相違

配布パッケージ名は **openqarp** ですが, Python コード内でインポートする際のトップレベルモジュール名は **qarp** となります.

```
1 $ pip install openqarp    # インストール時
```

```
1 import qarp                # Pythonスクリプト内での使用時
```

3 インストールと環境設定

3.1 動作要件

- Python バージョン: Python 3.11, 3.12, 3.13, 3.14
- 対応プラットフォーム: macOS (Apple Silicon M1/M2/M3/M4, Intel x86_64), Linux (Ubuntu/RHEL, x86_64, aarch64), Windows
- 前提ライブラリ: NumPy (≥ 1.24), SciPy (≥ 1.10), SymPy (≥ 1.12), NetworkX (≥ 3.0), Matplotlib

3.2 インストール手順

本研究の仮想環境 (`.venv`) 下において, 以下のコマンドでインストールを行います.

```

1 # 仮想環境のアクティベート
2 source .venv/bin/activate
3
4 # OpenQARP のインストール
5 pip install openqarp

```

3.3 インポートと ABI 適合性検証

OpenQARP は、C++ バックエンド (qarpx) と Python レイヤ (qarp) のバージョン整合性を保証するため、インポート時に自動的に ABI (Application Binary Interface) チェックを実行します。正しくセットアップされているか、以下のワンライナーで検証できます：

```

1 python -c "import qarp; print('OpenQARP version:', qarp.__version__)"
2 # 出力例: OpenQARP version: 0.1.0

```

4 コア機能と API リファレンス

4.1 ハミルトニアン定義：QubitOperator

OpenQARP の QubitOperator は、オープンソースの量子化学標準である OpenFermion と高い互換性を持ちつつ、内部計算がすべて C++ コア (qarpx) で実装されています。パウリ文字列と結合定数を直感的に定義できます。

```

1 from qarp.operators import QubitOperator
2
3 # 相互作用 Z_0 Z_1 (重み -2.5) と 局所磁場 Z_2 (重み 1.0) の定義
4 H_cost = QubitOperator("Z0 Z1", -2.5) + QubitOperator("Z2", 1.0)
5
6 # 定数項の追加 (エネルギーオフセット)
7 H_cost += QubitOperator("", 0.75)
8
9 print("コストハミルトニアン:")
10 print(H_cost)
11
12 # スパース行列 (scipy.sparse.csc_matrix) としての取り出しも瞬時に可能
13 H_sparse = H_cost.sparse_matrix(n_qubits=3)
14 print("行列サイズ:", H_sparse.shape)

```

Listing 1: QubitOperator による目的関数ハミルトニアンの定義

4.2 回路ビルディングブロック：Block と CompositeBlock

OpenQARP では、量子ゲートの集合を **Block** として扱います。複数のブロックを直列に連結することで **CompositeBlock** を構築します。

```

1 from qarp.blocks import CompositeBlock
2 from qarp.blocks._primitives import HnBlock, MixedOperatorBlock
3
4 n_qubits = 4
5
6 # 1. 全ビットへのアダマール初期化ブロック
7 init_block = HnBlock(n_qubits=n_qubits, target_qubits=list(range(n_qubits)))
8

```

```

9 # 2. Xミキサー層 (回転角  $\beta_0$  をシンボルに持つ標準ミキサー)
10 mixer_block = MixedOperatorBlock(n_qubits=n_qubits, symbol_idx=0)
11
12 # 3. 2つのブロックを合成
13 my_circuit = CompositeBlock(
14     n_qubits=n_qubits,
15     blocks=[init_block, mixer_block],
16     name="MyCustomCircuit"
17 )
18
19 print("含まれるシンボルパラメータ:", my_circuit.symbols)

```

Listing 2: CompositeBlock による回路のモジュール的組み立て

4.3 測定プリミティブと実行エンジン

アルゴリズムの実行形態（理論シミュレーションか、ショット数有限のサンプリングか）は、primitive オブジェクトによって切り替えます。

- `StateVector()`: 無雑音の厳密状態ベクトル計算（ショット数 ∞ の極限）。理論検証や勾配計算に最適。
- `Sampler(n_shots=1000)`: 実機測定を模したモンテカルロサンプリング。
- `Engine`: デフォルトの C++ エンジンに加え、`CudaqEngine` を指定することで GPU 高速化が可能。

5 OpenQARP による標準 QAOA の実装

OpenQARP の `qarp.algorithms.QAOA` は、変分量子アルゴリズム (VQA) フレームワークを継承しており、目的関数の定義から古典最適化ループの実行、最適パラメータの探索までを包括します。

5.1 QUBO 問題からの QAOA 実行

組合せ最適化問題を行列 Q (QUBO) から解く場合の最小完全コードを以下に示します。

```

1 import numpy as np
2 from qarp.algorithms import QAOA
3 from qarp.operators import QubitOperator
4 from qarp.optimizers import ScipyOptimizer
5 from qarp.algorithms._primitives import StateVector
6
7 # 1. 3量子ビットの QUBO 問題を定義:  $\min x^T Q x$ 
8 #  $H = -1.5 Z_0 Z_1 + 0.8 Z_1 Z_2 - 1.0 Z_0$ 
9 H_problem = (
10     QubitOperator("Z0 Z1", -1.5)
11     + QubitOperator("Z1 Z2", 0.8)
12     + QubitOperator("Z0", -1.0)
13 )
14
15 # 2. QAOA アルゴリズムインスタンスの生成 ( $p=1$  レイヤー)
16 # optimizer には Scipy の BFGS や COBYLA, CG が利用可能
17 qaoa = QAOA(
18     problem=H_problem,
19     n_layers=1,
20     optimizer=ScipyOptimizer("COBYLA", options={"maxiter": 50}),
21     primitive=StateVector(),

```

```

22     save_energy_history=True,
23     verbose=True
24 )
25
26 # 3. 最適化の実行 (古典オプティマイザが gamma, beta を探索)
27 result = qaoa.run()
28
29 print("===== 最適化結果 =====")
30 print(f"最小期待エネルギー: {result.optimal_value:.4f}")
31 print(f"最適パラメータ (gamma, beta): {result.optimal_parameters}")
32
33 # 4. 最適状態からの測定確率分布の抽出
34 state_probs = qaoa.get_statevector()
35 print("基底状態ベクトルの次元:", len(state_probs))

```

Listing 3: OpenQARP を用いた標準 QAOA の実行スクリプト

6 発展：制約付き最適化と「XY-QAOA」のカスタム構築

本研究の核心である「1-hot 制約付き材料最適化」を解くため、標準の X ミキサーではなく、励起数（粒子数）を保存する **XY ミキサー** ($e^{-i\beta(X_i X_j + Y_i Y_j)}$) を用いた **FM-XY-QAOA** を OpenQARP 上に構築します。

6.1 XY ミキサーブロックの設計数理

ブロックサイズ $K = 4$ (4 元素選択, 各 1-hot) の場合、ミキサーは各ブロック内で最近接リング状にスワップ演算子を適用します：

$$H_M^{XY} = \sum_{k=1}^M \sum_{j=1}^K \left(\sigma_{k,j}^x \sigma_{k,j+1}^x + \sigma_{k,j}^y \sigma_{k,j+1}^y \right) \quad (1)$$

OpenQARP の CompositeBlock を用いることで、このカスタムミキサーを数行で自作ブロックとして登録できます。

```

1  from typing import List
2  import numpy as np
3  from qarp.blocks import CompositeBlock
4  from qarp.blocks._primitives import CostOperatorBlock
5  from qarp.operators import QubitOperator
6  from qarp.algorithms._composite.vqa import VQA
7  from qarp.algorithms._primitives import StateVector
8  from qarp.optimizers import ScipyOptimizer
9  import sympy as sp
10
11 def build_xy_mixer_block(block_sizes: List[int], symbol_idx: int) -> CompositeBlock:
12     """各ブロック内で粒子数を保存するXYミキサーブロックを生成"""
13     beta = sp.Symbol(f"beta_{symbol_idx}")
14     n_qubits = sum(block_sizes)
15
16     # XX + YY 相互作用演算子を定義
17     H_xy = QubitOperator()
18     offset = 0
19     for sz in block_sizes:
20         for i in range(sz):
21             j = (i + 1) % sz # リング結合
22             q1, q2 = offset + i, offset + j

```

```

23         H_xy += QubitOperator(f"X{q1} X{q2}", 0.5)
24         H_xy += QubitOperator(f"Y{q1} Y{q2}", 0.5)
25         offset += sz
26
27     # パラメータ付きユニタリ発展ブロックとして返す
28     from qarp.blocks._primitives import TimeEvolutionBlock
29     return TimeEvolutionBlock(n_qubits=n_qubits, operator=H_xy, time=beta)
30
31 def build_one_hot_initial_state(block_sizes: List[int]) -> CompositeBlock:
32     """各ブロックの先頭ビットを |1> に反転させ、初期正当状態 |1000 1000 ...> を準備"""
33     from qarp.blocks._primitives import XBlock
34     n_qubits = sum(block_sizes)
35     blocks = []
36     offset = 0
37     for sz in block_sizes:
38         # 先頭 qubit (offset) に X ゲートを適用
39         blocks.append(XBlock(n_qubits=n_qubits, target_qubit=offset))
40         offset += sz
41     return CompositeBlock(n_qubits=n_qubits, blocks=blocks, name="OneHotInit")

```

Listing 4: OpenQARP 上での XY-QAOA カスタム実装

6.2 XY-QAOA クラスの完成

初期状態準備ブロックと各層の CostBlock, XYMixerBlock を連結することで、100% 制約を堅持する OpenQARP ネイティブな XYQAOA クラスが完成します。

```

1 class OpenQARP_XY_QAOA(VQA):
2     def __init__(self, cost_operator: QubitOperator, block_sizes: List[int], n_layers: int
3         = 1):
4         self.block_sizes = block_sizes
5         self.n_qubits = sum(block_sizes)
6         self.n_layers = n_layers
7
8         # 1. 1-hot 初期状態 |1000 1000 ...> の準備ブロック
9         circuit_blocks = [build_one_hot_initial_state(block_sizes)]
10
11        # 2. p 層の (Cost層 + XYミキサー層) の積層
12        for l in range(n_layers):
13            gamma = sp.Symbol(f"gamma_{l}")
14            from qarp.blocks._primitives import TimeEvolutionBlock
15            # Cost ハミルトニアンによる発展
16            circuit_blocks.append(TimeEvolutionBlock(self.n_qubits, cost_operator, time=
17                gamma))
18            # XY ミキサーによる発展
19            circuit_blocks.append(build_xy_mixer_block(block_sizes, symbol_idx=l))
20
21        full_ansatz = CompositeBlock(
22            n_qubits=self.n_qubits,
23            blocks=circuit_blocks,
24            name="XY_QAOA_Ansatz"
25        )
26
27        super().__init__(
28            operator=cost_operator,
29            ket=full_ansatz,
30            name="XY-QAOA",
31            optimizer=ScipyOptimizer("COBYLA", options={"maxiter": 30}),
32            primitive=StateVector()
33        )

```


7 BBO（触媒・機能性材料探索）への統合と今後の検証計画

7.1 既存 Qiskit 実装からのリプレイス手順

本リポジトリの `src/practical_materials_bbo.py` において、従来の Qiskit ソルバー `solve_xy_qaoa` を、上記の OpenQARP 実装 `solve_openqarp_xy_qaoa` に差し替えます。



図 1: OpenQARP を中核とする材料ブラックボックス最適化ループ

7.2 性能比較・ベンチマークのロードマップ

新規ブランチ `feature/openqarp-qaoa` において、以下の実証実験を順次進めます：

1. **実行速度比較:** $N = 8, 16, 20, 24$ において、Qiskit Aer シミュレータと OpenQARP C++ エンジンの計算時間を実測。
2. **解の等価性検証:** 同一のサロゲートハミルトニアンに対し、得られるエネルギー期待値および最適解出現確率が完全に一致することを検証。
3. **15 サイクル触媒 BBO の完走:** $N = 16$ (4 サイト $\times$ 4 元素) の多元触媒設計において、損失 0 円（制約充足率 100%）と真の最適解到達を OpenQARP 上で実証。

8 まとめ

富士通が公開した **OpenQARP** は、研究者が量子アルゴリズムの物理的・数学的本質に集中できるよう、卓越したモジュール性と C++ 高速コアを両立した最先端のフレームワークである。本研究で推進してきた FM-XY-QAOA の理論と OpenQARP の構成思想 (Composable Blocks) は完全に合致しており、本ガイドに示したアーキテクチャに沿って実装を移行することで、富士通インターンシップにおける研究成果の価値を最高水準へと高めることができる。